int considered harmful

A case against the default integer, and a type that fixes it

Peter Neiss

18 October 2026

Abstract

C++'s `int` is not a number. It is a platform-dependent bit pattern with undefined behaviour at both ends of its range. It changes width and signedness in the middle of expressions, it rounds without being asked, and it records nothing about what it counts. C++26 changes none of this. This paper puts `int` in the dock with eleven hazards, each a complete program with its real output. One of them makes GCC emit *no machine code at all* for `main`. The paper then shows, more briefly, that reaching for `double` trades one kind of silent wrongness for another.

It then presents `inside`, a header-only library (C++23 and later) with two layers. The number, `inside`, carries its range and step size in its type. The quantity layer, `units.hpp`, adds units, dimensions and quantity kinds on top of it, following Mateusz Pusz's mp-units. Arithmetic widens its result type at compile time, so overflow is unreachable. Division by zero is a value you can test. Fractions are exact rationals, a unit conversion is an exact rescale of the grid, a unit mismatch does not compile, and what happens at the edge of a range is a policy written in the declaration. Performance has no single answer. Against code that gives the same guarantee, `inside` is sometimes *faster*, sometimes byte-identical and sometimes substantially slower. We report all three, together with the programs that produced them.

A note on evidence. Every program in this paper is in `docs/int-considered-harmful/` in the `inside` repository. Every output block was captured by running it, and every measurement comes from those programs or from the repository's generated `docs/performance.md`. Every program is built as C++26 (`-std=c++26`), and every language rule cited is C++26's. The compiler is GCC 15.2.0 on x86-64 Linux unless stated otherwise. `inside` is at version 0.3 and still under development, so its API may change. The paper and the library were developed with Claude Code, which wrote and ran the programs, extracted the figures and typeset the document. The argument and the final review are the author's. The claims were checked against the compiler, not taken on the model's word, and every program is included so that you can check them too.

Contents

1 The thesis **3**

2 The indictment **3**

2.1 Signed overflow is undefined, and the optimiser believes it 3

2.2 The same assumption deletes an entire function 4

2.3 Two's complement is asymmetric 4

2.4 Division has two undefined inputs and no error value 5

2.5 Mixed signed/unsigned comparison 5

2.6 Small types do not stay small 6

2.7 Division truncates, and not in the direction you expect . . . 6
2.8 Shifts . . . 6
2.9 Narrowing is silent . . . 7
2.10 The midpoint bug . . . 7
2.11 The semantic hazard: int does not know what it counts . . . 8

3 "Just use double" 9
3.1 Decimal fractions are not representable . . . 9
3.2 The error accumulates . . . 9
3.3 Addition is not associative . . . 9
3.4 Integers stop counting at 2^{53} . . . 10
3.5 NaN breaks equality and ordering . . . 10

4 Why the usual defences fall short 10

5 inside: the range and the unit in the type 11
5.1 Arithmetic widens, so overflow is unreachable . . . 13
5.2 Division by zero is a value, not a signal . . . 14
5.3 Rounding is stated, not inherited . . . 14
5.4 The edge is a policy you declare . . . 15
5.5 Negation reflects the type . . . 15
5.6 The scale is part of the type . . . 16
5.7 The unit is part of the type . . . 17
5.8 Bounded indices . . . 18
5.9 No 64-bit ceiling . . . 19

6 A worked migration 19
6.1 An invoice . . . 19
6.2 A sensor reading . . . 21

7 Correctness 22
7.1 Errors caught before the program runs . . . 22
7.2 The error vocabulary . . . 23
7.3 Exactness . . . 23
7.4 Reproducibility . . . 24

8 Performance 24
8.1 The fast path is byte-identical to native . . . 24
8.2 The like-for-like comparison . . . 25
8.3 Where are the checks? . . . 26
8.4 The generated tables . . . 26
8.5 What correct rounding costs . . . 28

9 When not to use this 28

10 Conclusion 29

A Building the examples 29

B Program index 30

C References 30

1 The thesis

Dijkstra's 1968 letter did not say that `goto` produced wrong answers. It said that `goto` broke the link between the program on the page and the process it described. The case against `int` is narrower, and in a sense worse: `int` *does* produce wrong answers. It produces them silently, in code that compiles cleanly at maximum warning levels. It also produces them by letting the compiler *delete code the programmer wrote*, including the check written to catch that very failure.

The root is a mismatch of models. The programmer thinks of `int` as $\mathbb{Z}$. The standard defines it as a finite set with undefined behaviour at the boundary, and lets the optimiser assume the boundary is never reached. The two models agree where testing usually happens and disagree where it usually does not. The information needed to check an operation is: what range this value may occupy, what it counts, and what should happen if it leaves the range. That information lives only in the programmer's head, or in a comment.

`inside` puts that information in the type, where the compiler can act on it: the range and step size in the number, and the unit and kind in the quantity built on it. The rest of this paper does four things:

- shows that the problem is real (§2–§3);
- shows that the usual defences do not solve it (§4);
- shows that a type-level answer does (§5–§7);
- measures what that answer costs: sometimes nothing, sometimes a lot (§8).

2 The indictment

2.1 Signed overflow is undefined, and the optimiser believes it

Signed overflow does not wrap. It is undefined behaviour, so the compiler may assume it never happens and optimise on that assumption. Take a function that asks whether adding one to a number makes it bigger:

```
// Does adding 1 to an int always make it bigger?
[[gnu::noinline]] static bool grows(int x)
{
  return x + 1 > x;
}
```

`x + 1 > x` can be false only if `x + 1` overflows. Overflow is undefined, so the compiler treats the comparison as always true and deletes it, without a warning:

```
$ g++ -std=c++26 -O0 h01_overflow_ub.cpp -o h01 && objdump -d h01
0000000000001149 <grows(int)>:
    1149:   endbr64
    114d:   push   %rbp
    114e:   mov    %rsp,%rbp
    1151:   mov    %edi,-0x4(%rbp)     <- argument stored...
    1154:   mov    $0x1,%eax           <- ...and ignored. Return 1.
    1159:   pop    %rbp
    115a:   ret
```

Note the flag: `-O0`. GCC folds the comparison during translation, so a debug build does not protect you. No spelling of this check survives, because the check itself is what gets deleted.

```
INT_MAX            = 2147483647
grows(INT_MAX)     = yes, always
INT_MAX + 1        = -2147483648
```

The program reports that adding one always gives a larger value, and then prints the smaller value it got.

2.2 The same assumption deletes an entire function

The assumption compounds. Here is a loop whose counter starts near the top of the range:

```
int main()
{
  int count = 0;

  for (int i = INT_MAX - 2; i > 0; ++i)
  {
    ++count;
    if (count > 10)
    {
      std::printf("still going after %d iterations -- loop never terminates\n", count);
      break;
    }
  }

  std::printf("iterations = %d\n", count);
```

At `-O0` the counter wraps negative and the loop exits after three iterations. At `-O2` the compiler follows the chain to its end: `i > 0` cannot become false without overflow, so the loop is infinite, so nothing after it is reachable, so `main` has no reachable exit:

```
$ g++ -std=c++26 -O2 -Wall h02_overflow_loop.cpp -o h02
$ objdump -d h02

0000000000001040 <main>:
    1040:   endbr64
    1044:   cs nopw 0x0(%rax,%rax,1)
    104e:   xchg   %ax,%ax

0000000000001050 <_start>:
```

That is all of `main`: a landing pad and padding. There is no loop, no `printf` and **no `ret`**. Execution falls through into whatever comes next:

```
$ ./h02
Segmentation fault (core dumped)          [exit 139]

$ ./h02_compiled_at_O0
iterations = 3
```

`-Wall` said nothing. The same source and the same compiler at two optimisation levels give two results: one prints `3`, the other crashes before running a single instruction of the program. Undefined behaviour is a compile-time contract. Breaking it does not corrupt your result; it deletes your program.

2.3 Two's complement is asymmetric

There are 2^{31} negative `int` values and only $2^{31} - 1$ positive ones. At one point in the range, negation and absolute value have no correct answer.

```
  std::printf("INT_MAX           = %d\n", INT_MAX);
  std::printf("-INT_MIN          = %d   <- still negative\n", -m);
  std::printf("abs(INT_MIN)      = %d   <- still negative\n", std::abs(static_cast<int>(m)));
```

```
  // INT_MIN / -1 is also undefined: the true answer is INT_MAX+1.
  // On x86-64 it does not wrap, it raises SIGFPE. See h04_divzero.cpp.
```

```
INT_MIN            = -2147483648
INT_MAX            = 2147483647
-INT_MIN           = -2147483648   <- still negative
abs(INT_MIN)       = -2147483648   <- still negative
```

`std::abs` returns a negative number. That is not a library bug: there is no correct value it could return.

2.4 Division has two undefined inputs and no error value

Integer division has no `NaN`, so its two invalid inputs have no result to return. These are division by zero and `INT_MIN / -1`, whose true answer is `INT_MAX + 1`. On x86-64 both raise `SIGFPE`:

```
$ ./h04_divzero 0
about to evaluate 10 / 0 ...
Floating point exception (core dumped)          [exit 136]

$ ./h04_divzero 1
about to evaluate -2147483648 / -1 ...
Floating point exception (core dumped)          [exit 136]
```

The second case kills the process even though no zero is in sight. Code that divides by a runtime value must check two special cases, and almost none does.

2.5 Mixed signed/unsigned comparison

When an `int` meets an `unsigned`, both become unsigned, and a negative value becomes a huge positive one.

```
  // The classic: iterate backwards over an empty container.
  std::vector<int> v;   // empty
  std::printf("v.size()           = %zu\n", v.size());
  std::printf("v.size() - 1       = %zu   <- not -1\n", v.size() - 1);

  int visited = 0;
  for (std::size_t i = 0; i < v.size() - 1; ++i)
  {
    ++visited;
    if (visited > 3) break;   // escape hatch: this loop runs ~2^64 times
  }
  std::printf("loop body entered  = %d times on an EMPTY vector\n", visited);
```

```
-1 < 0u            = false
v.size()           = 0
v.size() - 1       = 18446744073709551615   <- not -1
loop body entered  = 4 times on an EMPTY vector
```

`-Wall` warns about the literal `-1 < 0u`, but not about the loop, where both sides are unsigned. Without the escape hatch, the loop over the empty vector would run about 1.8×10^{19} times and index past the allocation on its first pass. This is a common source of exploitable memory-safety bugs, and it comes from a *comparison*.

2.6 Small types do not stay small

```
  std::uint8_t a = 200;
  std::uint8_t b = 200;

  // Both operands are uint8_t; the product is not.
  std::printf("uint8 * uint8 -> int? %s\n",
              std::is_same_v<decltype(a * b), int> ? "yes" : "no");
  std::printf("200 * 200          = %d\n", a * b);
  std::printf("stored back in u8  = %u   <- truncated\n",
              static_cast<std::uint8_t>(a * b));

  // char signedness is not specified by the standard.
  std::printf("char is signed?    %s (platform-dependent)\n",
              std::is_signed_v<char> ? "yes" : "no");

  // Widths are minimums, not guarantees.
  std::printf("sizeof(int)        = %zu bytes here; the standard promises >= 2\n",
              sizeof(int));
```

```
uint8 * uint8 -> int? yes
200 * 200          = 40000
stored back in u8  = 64   <- truncated
char is signed?    yes (platform-dependent)
sizeof(int)        = 4 bytes here; the standard promises >= 2
```

This output shows three portability hazards:

- The 8-bit product is computed in `int`, and the store truncates it.
- Whether bare `char` is signed depends on the platform.
- `int` is 2 bytes on plenty of shipping embedded hardware.

2.7 Division truncates, and not in the direction you expect

```
  std::printf(" 7 / 2             = %d   (exact answer 3.5)\n", 7 / 2);
  std::printf("-7 / 2             = %d   (rounds toward zero, not down)\n", -7 / 2);
  std::printf(" 7 %% 2             = %d\n", 7 % 2);
  std::printf("-7 %% 2             = %d   <- negative remainder\n", -7 % 2);

  // Averaging three test scores: 90, 90, 91 -> 90, not 90.33
  int total = 90 + 90 + 91;
  std::printf("mean(90,90,91)     = %d   (true mean 90.333...)\n", total / 3);
```

```
 7 / 2             = 3   (exact answer 3.5)
-7 / 2             = -3   (rounds toward zero, not down)
 7 % 2             = 1
-7 % 2             = -1   <- negative remainder
mean(90,90,91)     = 90   (true mean 90.333...)
```

Rounding toward zero is neither floor nor nearest. It is the only mode on offer, it is applied silently, and it is wrong for averages, bill splitting and percentages. Because the remainder is negative for negative operands, `x % n == k` fails as an index into a cyclic structure.

2.8 Shifts

```
  volatile int one   = 1;
```

```
  volatile int shift = 31;

  // Defined since C++20: 1 * 2^31 modulo 2^32. A positive value turns negative.
  std::printf("1 << 31            = %d   <- defined, and negative\n", one << shift);

  // Defined since C++20: >> rounds down, / rounds toward zero.
  std::printf("-7 >> 1            = %d\n", -7 >> 1);
  std::printf("-7 / 2             = %d   <- same \"halve\", other answer\n", -7 / 2);

  // Still undefined: a count >= the width (or negative). x86 masks the
  // count to 5 bits, so this prints 1 here; the standard promises nothing.
  shift = 32;
  std::printf("1 << 32            = %d   <- undefined behaviour\n", one << shift);
  return 0;
```

```
1 << 31            = -2147483648   <- defined, and negative
-7 >> 1            = -4
-7 / 2             = -3   <- same "halve", other answer
1 << 32            = 1   <- undefined behaviour
```

C++20 made signed shifts well-defined. « now wraps modulo 2^N, and » on a negative value rounds down. Defined does not mean right: a positive value shifted into the sign bit silently turns negative, and halving by » and halving by `/` round in different directions. A shift count that is negative, or not less than the width, is still undefined. x86 masks the count, so `1 « 32` printed `1` here. The standard promises nothing, and only UBSan reported it.

2.9 Narrowing is silent

```
int main()
{
  std::int64_t horizontal_velocity = 40000;   // fine on the old rocket

  std::int16_t converted = static_cast<std::int16_t>(horizontal_velocity);

  std::printf("velocity (64-bit)  = %ld\n", horizontal_velocity);
  std::printf("stored as 16-bit   = %d   <- sign flipped\n", converted);

  // Implicit narrowing compiles without a cast, and without a warning
  // unless you asked for one.
  int  big   = 300;
  char small = big;
  std::printf("int 300 -> char    = %d\n", small);
```

```
velocity (64-bit)  = 40000
stored as 16-bit   = -25536   <- sign flipped
int 300 -> char    = 44
```

Since C++20 these conversions are defined: the value is reduced modulo 2^N. Defined is not the same as right, and nothing warns. This is Ariane 5 Flight 501 in miniature. In June 1996, a horizontal velocity was converted to a 16-bit signed integer by software carried over from Ariane 4, where the value had been provably smaller. The conversion overflowed 37 seconds after ignition and the vehicle was lost. The code was correct for the range it was written against, but nothing in the type recorded that range.

2.10 The midpoint bug

```
int main()
{
  volatile int lo = INT_MAX - 3;
  volatile int hi = INT_MAX - 1;

  int mid_broken = (lo + hi) / 2;          // overflows
  int mid_safe   = lo + (hi - lo) / 2;     // does not

  std::printf("lo                 = %d\n", lo);
  std::printf("hi                 = %d\n", hi);
  std::printf("(lo + hi) / 2      = %d   <- outside [lo, hi]\n", mid_broken);
  std::printf("lo + (hi - lo) / 2 = %d   <- correct\n", mid_safe);
```

```
lo                 = 2147483644
hi                 = 2147483646
(lo + hi) / 2      = -3   <- outside [lo, hi]
lo + (hi - lo) / 2 = 2147483645   <- correct
```

The midpoint of two in-range values lands outside the range. Bentley's *Programming Pearls* (1986) has this bug, and Bloch found it again in 2006 in the JDK's `Arrays.binarySearch`, where it had lived for nine years. The problem is not a lack of care.

2.11 The semantic hazard: int does not know what it counts

Every hazard so far concerned range. This one concerns meaning, and no sanitizer can catch it, because the arithmetic itself is valid.

```
int price_in_cents(int item)
{
  static const int table[] = {1999, 450, 1234};
  return table[item];
}

int shipping_in_dollars()
{
  return 5;   // $5 flat rate
}

int main()
{
  int subtotal = price_in_cents(0) + price_in_cents(1) + price_in_cents(2);
  int total    = subtotal + shipping_in_dollars();   // <- units mismatch

  std::printf("subtotal (cents)   = %d\n", subtotal);
  std::printf("shipping (dollars) = %d\n", shipping_in_dollars());
  std::printf("total              = %d   <- charged 5 cents for shipping\n", total);
  std::printf("compiles clean, runs clean, bills the customer wrong.\n");
```

```
subtotal (cents)   = 3683
shipping (dollars) = 5
total              = 3688   <- charged 5 cents for shipping
compiles clean, runs clean, bills the customer wrong.
```

Cents and dollars have the same type, so adding them is well-defined, warning-free and wrong. The Mars Climate Orbiter was lost in 1999 to this class of error: pound-force seconds where newton seconds were expected. No value was out of range. What the number *meant* was never written down anywhere the compiler could see it.

3 “Just use double”

The usual answer to the above is floating point: a wider range, no trap on division by zero, and fractions. `double` is the right tool for many problems, but it does not fix these ones. It brings its own family of silent wrongness. This section is shorter only because those failures are better known.

3.1 Decimal fractions are not representable

```
0.1 + 0.2          = 0.30000000000000004441
0.3                = 0.29999999999999998890
0.1 + 0.2 == 0.3   ? false
```

3.2 The error accumulates

```
int main()
{
  double total = 0.0;
  for (int i = 0; i < 10000; ++i)
    total += 0.01;   // one cent, ten thousand times

  std::printf("10000 x $0.01      = %.20f\n", total);
  std::printf("expected           = %.20f\n", 100.0);
  std::printf("exactly $100.00    ? %s\n", (total == 100.0) ? "true" : "false");
  std::printf("error              = %.20f\n", total - 100.0);
```

```
10000 x $0.01      = 100.00000000001425348728
expected           = 100.00000000000000000000
exactly $100.00    ? false
error              = 0.00000000001425348728
```

After ten thousand transactions the ledger no longer balances to the cent. The error is small enough to survive testing, and then it shows up in reconciliation.

3.3 Addition is not associative

```
int main()
{
  double a = 1e16;
  double b = -1e16;
  double c = 1.0;

  std::printf("(a + b) + c        = %.1f\n", (a + b) + c);
  std::printf("a + (b + c)        = %.1f\n", a + (b + c));
  std::printf("equal              ? %s\n",
              (((a + b) + c) == (a + (b + c))) ? "true" : "false");
```

```
(a + b) + c        = 1.0
a + (b + c)        = 0.0
equal              ? false
```

The same three values give two answers depending on grouping. Change the thread count of a parallel reduction, or let the optimiser re-associate a loop, and the sum changes. Results are then not reproducible even from run to run.

3.4 Integers stop counting at 2^{53}

```
int main()
{
  double x = 9007199254740992.0;   // 2^53

  std::printf("2^53               = %.1f\n", x);
  std::printf("2^53 + 1           = %.1f   <- unchanged\n", x + 1.0);
  std::printf("2^53 + 1 == 2^53   ? %s\n", (x + 1.0 == x) ? "true" : "false");

  // Catastrophic cancellation: subtracting near-equal values destroys
  // every significant digit that mattered.
  double big = 1e16;
  std::printf("(1e16 + 1) - 1e16  = %.1f   (should be 1)\n", (big + 1.0) - big);
```

```
2^53               = 9007199254740992.0
2^53 + 1           = 9007199254740992.0   <- unchanged
2^53 + 1 == 2^53   ? true
(1e16 + 1) - 1e16  = 0.0   (should be 1)
```

Past 2^{53} a counter stops incrementing. That matters for identifiers and timestamps that pass through JSON. The last line is catastrophic cancellation: subtracting two nearly equal values wipes out every digit that mattered.

3.5 NaN breaks equality and ordering

```
0.0 / 0.0          = -nan
1.0 / 0.0          = inf   <- no error reported
nan == nan         ? false   <- breaks equality
nan < 1            ? false
nan > 1            ? false   <- neither; breaks sorting
nan + 1000         = -nan   <- contaminates the whole total
```

An invalid operation does not trap. It yields a value that spreads silently through every later computation. That value also breaks the strict weak ordering `std::sort` requires, so sorting a range that contains a `NaN` is undefined behaviour. Floating point is excellent for continuous quantities over a huge dynamic range. It is a poor fit for money, counting, identifiers and anything that must reproduce.

4 Why the usual defences fall short

Sanitizers are runtime instruments. UBSan detects overflow only when it happens, and only in an instrumented build. It cannot see what the optimiser already deleted. On our first example:

```
$ g++ -std=c++26 -O2 -fsanitize=undefined h01_overflow_ub.cpp && ./a.out
h01_overflow_ub.cpp:24:14: runtime error: signed integer overflow:
    2147483647 + 1 cannot be represented in type 'int'
INT_MAX            = 2147483647
grows(INT_MAX)     = yes, always
INT_MAX + 1        = -2147483648
```

It reports line 24, the `printf` argument. It says nothing about `grows()`, whose comparison was already folded away, so there was no arithmetic left to instrument.

-fwrapv changes the language. It removes the undefined behaviour and makes the wrong answer reliable: the midpoint is still negative. It fixes the UB, not the arithmetic.

Contracts and hardening are runtime checks. C++26 contracts (`pre`, `post`, `contract_assert`) put a precondition in the declaration, and a hardened standard library traps an out-of-bounds `v[i]`. Both check at runtime, only where someone wrote the predicate or called the library. A contract's evaluation semantic may be `ignore`, and then nothing is checked. Neither changes what `a + b` does.

The standard's helpers fix one call at a time. `std::midpoint`, `std::cmp_less` and `std::in_range` (C++20) fix the midpoint and the mixed comparison. C++26 adds saturation (`std::add_sat`, `std::saturate_cast`) and `<stdckdint.h>`, whose `ckd_add` reports overflow. Each works only at the call sites that remember to use it, and only for the full range of a built-in type.

Static analysis is partial. Range analysis across functions and data structures is undecidable in general. No analyser can tell that one `int` holds cents and another holds dollars, because nothing in the program says so.

Unit libraries solve the other hazard. mp-units, by Mateusz Pusz, and Boost.Units check dimensions at compile time, and mp-units is proposed for the standard as P3045. They catch §2.11. But their number is still an `int` or a `double`, with every hazard of §2 and §3: with an `int`, km to mm can overflow, and with a `double`, inches to millimetres rounds.

Safe-integer libraries solve one hazard. `boost::safe_numerics` and `SafeInt` fix overflow (§2.1–§2.2). They do not handle units, rounding policy or ranges narrower than the underlying type. They also check at runtime what could have been rejected at compile time.

Every one of these tools works at a point where the necessary information has already been thrown away, or records only part of it: the range, the unit, the rounding rule and the behaviour at the boundary. The proposal is simply to record all of it.

5 inside: the range and the unit in the type

`inside` is a header-only library with two layers. It needs C++23; C++26 adds grids past 64 bits and diagnostics that show values.

The number. `inside` is a number whose type holds its grid and its edge policy:

```
template<grid G, policy_flag P>
struct inside;                          // include/beman/inside/core.hpp:98
```

The `grid` is a compile-time value with two parts: an interval, and a *notch*, which is the step size held as an exact rational. Every value is exactly Lower + $k \cdot$ Notch for some integer k. The `policy` says what happens when a value would leave the range. The grid alone determines the storage:

- the narrowest of `uint8`…`int64` that can number every point;
- an exact fraction for a continuous grid;
- a wider integer for grids past 64 bits (§5.9).

The quantity. `units.hpp` gives a number a unit and a kind. The number is always an `inside`:

```
template<reference R, insidable Rep>
class quantity;                       // include/beman/inside/units.hpp:1180
template<reference R, origin_ref O, insidable Rep>
class quantity_point;                 // include/beman/inside/units.hpp:1718
```

A reference is either a unit, such as `si::kilo(si::metre)`, or a quantity kind with a unit, such as `isq::height[si::metre]`. A quantity point counts from an origin, such as absolute zero or the ice point. `units_math.hpp` adds the math functions on quantities.

Units of your own are values too: here a per cent, a currency with euros and cents, and samples per second. The catalogue supplies the degree, the radian, the kilometre and the height kind:

```
// Units of our own: per cent, a currency, samples per second.
inline constexpr named_unit     percent_def{"%", mag<rational{1, 100}> * one,
                                           {.space_before = false}};
inline constexpr auto           percent = unit_c<percent_def>;
inline constexpr base_dimension currency_def{"currency"};
inline constexpr quantity_spec  amount_def{"amount_of_money", currency_def};
inline constexpr named_unit     euro_def{"EUR", spec_c<amount_def>};
inline constexpr auto           euro = unit_c<euro_def>;
inline constexpr named_unit     cent_def{"ct", mag<rational{1, 100}> * euro};
inline constexpr auto           cent = unit_c<cent_def>;
inline constexpr named_unit     sps_def{"SPS", one / si::second, {.kind = isq::frequency}};
inline constexpr auto           sps = unit_c<sps_def>;

using pct         = quantity<percent, inside<{0, 100}>>;
using money       = quantity<euro, inside<{{0, 1'000'000}, per<100>}, round_nearest>>;
using angle       = quantity<si::degree, inside<{0, 359}, wrap>>;   // modular
using rad_number  = inside<{{0, 7}, per<10'000>}, round_nearest>;
using deg_number  = inside<{{0, 360}, per<100>}, round_nearest>;
using sample_rate = quantity<sps, inside<{0, 192'000}>>;
using trip        = quantity<si::kilo(si::metre), inside<{{0, 500}, per<1000>}>>;
using height      = quantity<isq::height[si::metre], inside<{0, 9000}>>;

int main()
{
  std::cout << "pct          = " << pct{42} << "\n";
  money price{19.99};
  std::cout << "money        = " << price << " = " << price.in(cent) << "\n";
  std::cout << "angle        = " << angle{370} << "   (370° wraps)\n";
  // Degrees <-> radians is a factor of pi: exact, then rounded once.
  auto rad = value_cast<si::radian, rad_number>(angle{90});
  auto deg = value_cast<si::degree, deg_number>(quantity<si::radian, inside<{0, 6}>>{1});
  std::cout << "90° in rad   = " << rad << "\n";
  std::cout << "1 rad in deg = " << deg << "\n";
  std::cout << "sample_rate  = " << sample_rate{48'000} << "\n";
  std::cout << "trip         = " << trip{12.5} << " = " << trip{12.5}.in(si::metre) << "\n";
  std::cout << "height       = " << height{8849} << "\n";
```

```
pct          = 42%
money        = 19.99 EUR = 1999 ct
angle        = 10°   (370° wraps)
90° in rad   = 1.5708 rad
1 rad in deg = 57.30°
sample_rate  = 48000 SPS
trip         = 12.500 km = 12500 m
height       = 8849 m
```

Euros to cents is an exact rescale. Degrees to radians is a factor of π, which no grid holds exactly. `.in()` refuses it, and `value_cast` from `units_math.hpp` computes $x\pi/180$ exactly and rounds once onto the target grid.

Credit where it is due. The quantity layer follows **mp-units**, the library by **Mateusz Pusz** and the basis of the WG21 proposal P3045, *Quantities and units library*. The design is mp-units': the ISQ tree of quantity kinds, references as unit plus kind, the conversion rules between kinds, and quantity points with origins. `units.hpp` reimplements that model with an `inside` as the number and does not depend on mp-units. Three things differ:

- the representation is always an `inside`: no `double` or `int` representations;
- conversions and mixed-unit sums are exact, and their result grids are inferred;
- a factor of π scales the unit and rounds once, only when asked.

5.1 Arithmetic widens, so overflow is unreachable

The result type of an operation is computed at compile time from the operand grids. The sum of two values in $[10, 255]$ has the range $[20, 510]$, which no input can overflow. There is no overflow to be undefined and nothing to check.

```cpp
int main()
{
  using u8 = inside<{10, 255}>;
  constexpr u8 a{16};
  constexpr u8 b{220};

  // The sum of two [10,255] values lives in [20,510]. Checked at compile time.
  static_assert(a + b == 236);
  std::cout << "a + b              = " << (a + b) << "\n";

  // A difference that would underflow a native uint8 widens to signed instead.
  static_assert(a - b == -204);
  std::cout << "a - b              = " << (a - b) << "   (no wraparound)\n";

  // The midpoint computation that breaks binary search cannot break here:
  // lo + hi is computed in a grid wide enough to hold it.
  using idx = inside<{0, 2147483647}>;
  idx lo{2147483644};
  idx hi{2147483646};
  auto mid = (lo + hi) / just<2>;   // a plain value: 2 is not zero, and it fits
  std::cout << "(lo + hi) / 2      = " << mid << "   (still in range)\n";
```

```
a + b              = 236
a - b              = -204   (no wraparound)
(lo + hi) / 2      = 2147483645   (still in range)
```

The listing shows three things:

- `static_assert` verified the sum, so a wrong answer would fail the build, not a test.
- The difference of two unsigned-ranged values has the grid $[-245, 245]$. It widened to a signed type instead of wrapping.
- The midpoint from §2.10 stayed in range, because `lo + hi` was computed in a grid wide enough to hold it.

5.2 Division by zero is a value, not a signal

Division returns a plain value when two things hold: the divisor's grid excludes zero, and the exact quotient provably fits. The operand grids decide both at compile time, and every integer grid satisfies the second. Otherwise division returns `std::expected<inside, errc>`.

```
int main()
{
  using val = inside<{0, 100}>;

  val a = 7;
  val b = 3;

  // Exact rational result -- no silent truncation.
  auto exact = a / b;
  if (exact)
    std::cout << "7 / 3              = " << *exact << "   (exact, not 2)\n";

  // Opt in to C-style truncation explicitly, per call.
  auto trunc = div(a, b, rounded_toward_zero);
  if (trunc)
    std::cout << "div(7, 3, trunc)   = " << *trunc << "\n";

  // Division by zero: an error value, and the program keeps running.
  val zero = 0;
  auto oops = val(10) / zero;
  std::cout << "10 / 0             = "
            << (oops.has_value() ? "value" : errc_message(oops.error()))
            << "   (no SIGFPE)\n";
```

```
7 / 3              = 7/3   (exact, not 2)
div(7, 3, trunc)   = 2
10 / 0             = division by zero   (no SIGFPE)
```

By default the quotient is *exact*: 7/3, not 2. Truncation is available, but you ask for it by name. The divisor's grid $[0, 100]$ contains zero, so the result is an `expected`, and the failure that killed the process in §2.4 becomes a value you can test.

5.3 Rounding is stated, not inherited

```
  using val = inside<{-100, 100}>;
  using pos = inside<{1, 100}>;   // divisor grid excludes zero

  val n{-7};
  pos d{2};

  // The default is the exact quotient. The divisor cannot be zero and the
  // quotient provably fits, so it is a plain value: nothing to unwrap.
  std::cout << "n / d               = " << n / d << "   (exact)\n";

  // Or, if you ask for a rounding, the one you name -- per call.
  std::cout << "rounded_nearest     = " << div(n, d, rounded_nearest) << "\n";
  std::cout << "rounded_floor       = " << div(n, d, rounded_floor) << "\n";
  std::cout << "rounded_ceil        = " << div(n, d, rounded_ceil) << "\n";
  std::cout << "rounded_toward_zero = " << div(n, d, rounded_toward_zero) << "\n";
```

```
n / d               = -3.5   (exact)
rounded_nearest     = -4
```

```
rounded_floor       = -4
rounded_ceil        = -3
rounded_toward_zero = -3
```

`pos` excludes zero, so the exact quotient is a plain value. Each named mode is honoured for negative operands, where truncation and flooring differ. A property test pins $(a/b) \cdot b + a \bmod b = a$ across whole grids, in every mode and with negative divisors (`tests/beman/inside/div_round.test.cpp:150`).

5.4 The edge is a policy you declare

Arithmetic never leaves its range. Only assignment into a narrower type can, and what happens then is part of the declaration:

```cpp
int main()
{
  // checked (the default): reports rather than storing a wrong value.
  try
  {
    checked_100 x = 200;
    (void)x;
  }
  catch (beman::inside::inside_error& e)
  {
    std::cout << "checked  200 ->    " << errc_message(e.Code) << "\n";
  }

  // clamp: saturate at the boundary.
  clamp_100 cl = 150;
  std::cout << "clamp    150 ->    " << cl << "\n";

  // wrap: modular arithmetic, on purpose this time.
  wrap_360 wr = 370;
  std::cout << "wrap     370 ->    " << wr << "\n";

  // try_make: failure is a value you test, not an exception.
  auto tm = digit::try_make(10);
  std::cout << "try_make  10 ->    " << (tm ? "has value" : errc_message(tm.error())) << "\n";

  // Per-operation override on an otherwise-checked value.
  checked_100 p{50};
  p.with_clamp() = 150;
  std::cout << "with_clamp() 150 -> " << p << "\n";
```

```
checked  200 ->    value does not fit its range
clamp    150 ->    100
wrap     370 ->    10
try_make  10 ->    value does not fit its range
with_clamp() 150 -> 100
```

Wrapping is available, but here you request it, and it is well-defined. The default is `checked`, and every policy checks unless it says `unsafe`. For example, `round_nearest` rounds first and then still range-checks the result. To turn the check off, you have to write the word.

5.5 Negation reflects the type

Negating a [Lower, Upper] value gives a [−Upper, −Lower] value, so the asymmetry of §2.3 cannot arise:

```
int main()
{
  using temp = inside<{-40, 125}>;   // an asymmetric range, on purpose

  constexpr temp coldest{-40};

  // -coldest has type inside<{-125, 40}> -- the interval is mirrored.
  static_assert(-coldest == 40);
  std::cout << "coldest            = " << coldest << "\n";
  std::cout << "-coldest           = " << (-coldest) << "   (representable by
    construction)\n";

  // The same holds at the extreme of the range: there is no value whose
  // negation falls outside the negated type.
  constexpr temp hottest{125};
  static_assert(-hottest == -125);
  std::cout << "-hottest           = " << (-hottest) << "\n";
```

```
coldest            = -40
-coldest           = 40   (representable by construction)
-hottest           = -125
```

`-INT_MIN` has no analogue, and for the same reason `abs` cannot return a negative number.

5.6 The scale is part of the type

The notch records the scale, so a cent value and a dollar value have different types, and combining them converts:

```
using money   = inside<{{0, 1'000'000}, per<100>}, round_nearest>;
using dollars = inside<{0, 1000}>;   // whole dollars only

int main()
{
  money subtotal{0};
  subtotal += money{19.99};
  subtotal += money{4.50};
  subtotal += money{12.34};
  std::cout << "subtotal           = $" << subtotal << "\n";

  dollars shipping{5};   // "5" -- but 5 what? The type says: whole dollars.
  std::cout << "shipping           = $" << shipping << "\n";

  // Adding them scales the dollar value onto the cent grid automatically.
  auto total = subtotal + shipping;
  std::cout << "total              = $" << total << "   (not $36.88)\n";

  // A bare integer cannot enter the arithmetic at all: `subtotal + 5` is
  // ill-formed, because 5 carries no grid. See b06_compile_time.cpp.
```

```
subtotal           = $36.83
shipping           = $5
total              = $41.83   (not $36.88)
```

A bare integer cannot enter the arithmetic at all, because nothing says which grid it belongs to. The diagnostic says so:

```
$ g++ -std=c++26 -I ../../include x01_bare_int_rejected.cpp
   16 |   auto bad = x + 1;      // error: no grid for the literal
```

```
include/beman/inside/arithmetic.hpp:552:31: error: static assertion failed:
   an inside cannot be combined with a raw scalar: give the scalar a grid --
   `1_ins`, `just<1>`, or `inside<{lo,hi}>{n}` for a runtime value with a
   known range
```

5.7 The unit is part of the type

Scale is not the same as dimension. A quantity carries its unit, so the Mars Climate Orbiter mismatch of §2.11 converts exactly, and adding an impulse to a length does not compile:

```
constexpr auto lbf_s = international::pound_force * si::second;
constexpr auto n_s   = si::newton * si::second;

template <class A, class B>
concept addable = requires(A a, B b) { a + b; };

int main()
{
  quantity<lbf_s, inside<{0, 100}>> reported{10};   // ground software: lbf·s
  quantity<n_s, inside<{0, 1000}>>  budget{100};    // navigation: N·s

  // The sum runs in a common unit; both conversions are exact rescales.
  auto total = budget + reported;
  std::cout << "100 N·s + 10 lbf·s = " << total.in(n_s) << "\n";

  // An impulse and a length do not add: the expression is ill-formed.
  quantity<si::metre, inside<{0, 1000}>> d{5};
  static_assert(!addable<decltype(budget), decltype(d)>);
  std::cout << "N·s + m            : does not compile\n";
```

```
100 N·s + 10 lbf·s = 144.4822161526050 N s
N·s + m            : does not compile
```

A sum runs in the common unit of its operands. Both sides are rescaled exactly, and then the `inside` addition runs, so a mixed-unit sum cannot overflow. A rejected call states its reason as a deleted overload: `units:  adding or comparing quantities of different dimensions`.

Units do more than dimensions. A conversion multiplies the grid, not the number. Temperatures are points with origins. Kinds separate quantities that share a dimension:

```
  // A conversion multiplies the grid, not the number: same raw, exact.
  quantity<si::kilo(si::metre), inside<{{0, 500}, per<1000>}>> trip{12.5};
  auto metres = trip.in(si::metre);
  std::cout << "12.5 km in m       = " << metres << "   (raw "
            << trip.numerical_value().raw() << " -> " << metres.numerical_value().raw() <<
    ")\n";

  // A temperature is a point; its origin moves by an exact offset.
  auto room = point<si::degree_Celsius>(inside<{{-50, 50}, per<10>}>{20});
  std::cout << "20 C in K          = " << room.point_for(si::absolute_zero).in(si::kelvin) <<
    "\n";

  // One dimension, two kinds: hertz and becquerel are both 1/s.
  quantity<si::hertz, inside<{0, 1000}>>     f{50};
  quantity<si::becquerel, inside<{0, 1000}>> a{50};
  static_assert(!addable<decltype(f), decltype(a)>);
  std::cout << "50 Hz + 50 Bq      : does not compile\n";

  // A width times a height is an area.
```

```
  quantity<isq::width[si::metre], inside<{0, 10}>>  w{3};
  quantity<isq::height[si::metre], inside<{0, 10}>> h{4};
  quantity<isq::area[square(si::metre)], inside<{0, 100}>> floor_area = w * h;
  std::cout << "3 m x 4 m          = " << floor_area << "\n";

  // A ratio of two lengths is dimensionless: it converts to a plain inside.
  quantity<si::kilo(si::metre), inside<{1, 10}>> run{3};
  quantity<si::metre, inside<{1, 10}>>           step{2};
  inside<{0, 10000}, round_nearest> ratio = run / step;
  std::cout << "3 km / 2 m         = " << ratio << "\n";
```

```
12.5 km in m       = 12500 m   (raw 12500 -> 12500)
20 C in K          = 293.15 K
50 Hz + 50 Bq      : does not compile
3 m x 4 m          = 12 m^2
3 km / 2 m         = 1500
```

- **Conversion keeps the raw.** km to m turns the grid's notch of 1/1000 km into 1 m, and the stored integer does not change. At runtime a conversion is a `mov` (§8.1).
- **Points change origin exactly.** 20 °C sits 273.15 K above absolute zero. The grid shifts by that offset, with no rounding.
- **Kinds keep one dimension apart.** Hz and Bq are both s^{-1}, but a frequency is not an activity: `units:  adding or comparing quantities of different kinds`. A width times a height, by contrast, is an area.
- **A ratio is a number.** 3 km / 2 m has dimension one and converts to a plain `inside`.

Units, kinds and references are `consteval` values, not type lists for the compiler to sort, so `m·s` and `s·m` are one unit. A factor of π (degrees to radians) scales the unit instead of the number, so π is never approximated; `value_cast` rounds it once, with the correctly rounded engine of §7.

5.8 Bounded indices

```
int main()
{
  std::array<int, 10> data{};
  for (std::size_t i = 0; i < data.size(); ++i)
    data[i] = static_cast<int>(i * i);

  using index = inside<{0, 9}>;

  // In range: usable directly.
  index i{7};
  std::cout << "data[7]            = " << data[i.to<std::size_t>().value()] << "\n";

  // Out of range: an error value, not a buffer overrun.
  auto bad = index::try_make(10);
  std::cout << "index 10           = " << (bad ? "has value" : errc_message(bad.error()))
            << "   (no out-of-bounds access possible)\n";

  // Iterating the valid slots exactly, with no off-by-one to get wrong.
  int sum = 0;
  for (auto k : inside_range<{0, 9}>{})
    sum += data[k.to<std::size_t>().value()];
  std::cout << "sum of all slots   = " << sum << "\n";
```

```
data[7]            = 49
index 10           = value does not fit its range   (no out-of-bounds access possible)
sum of all slots   = 285
```

The bounds check runs once, at construction. An index that exists is in range, and an index that would not be in range is never created.

5.9 No 64-bit ceiling

Without static reflection, the limits and notch of a grid are 64-bit fractions. With C++26 static reflection (GCC 16, `-freflection`) they have no size limit:

- large grid numbers are interned in static storage (`std::define_static_array`), so equal grids are the same type however they are spelled;
- the raw storage becomes a fixed-width integer sized from the grid.

```
  // int64 stops at 9223372036854775807. A [0, 2^100] grid does not.
  using huge = inside<{0, 0x1p100}>;
  constexpr huge a = 1e30;                    // the double 1e30, an integer, held exactly

  // a * a lives in [0, 2^200]: the product grid is computed at compile time,
  // so the multiplication cannot overflow and returns a plain value.
  constexpr auto sq = a * a;
  std::cout << "a                  = " << a << "\n";
  std::cout << "a * a              = " << sq << "\n";
  std::cout << "sizeof(a), sizeof(a * a) = " << sizeof(a) << ", " << sizeof(sq) << " bytes\n";

  // A notch of exactly 10^-30, which no double and no int64 ratio can hold.
  using fine = inside<{{0, 1}, 1e-30_g}, round_nearest>;
  fine third = just<1, 3>;                    // rounded once, onto the 1e-30 grid
  std::cout << "1/3 on a 1e-30 grid = " << third << "\n";
```

```
$ g++-16 -std=c++26 -freflection -O2 -I ../../include b11_no_ceiling.cpp
a                  = 1000000000000000019884624838656
a * a              = 1000000000000000039769249677312000395398304974095154031886336
sizeof(a), sizeof(a * a) = 16, 32 bytes
1/3 on a 1e-30 grid = 0.333333333333333333333333333333
```

The first line is not a typo. The literal `1e30` is a `double`, and that is its exact value; an exact 10^{30} is spelled `1e30_g`. The product lives in $[0, 2^{200}]$, so the widening rule still applies: the result is a plain 32-byte value, with no allocation. No `double` and no 64-bit ratio can hold the notch 10^{-30}. Compiled without reflection, the file is rejected with a message naming the C++26 big-grid requirement. Big grids also complete the unit catalogue: the SI prefixes past $10^{\pm 18}$, the eV, and the exact defining constants h, e, k and N_A, so $E = h\nu$ is exact.

6 A worked migration

6.1 An invoice

Here is an invoice in `int`. It has three defects, and the compiler mentions none of them.

```
int main()
{
  // A large but entirely plausible invoice: $2,684,354.56 in cents.
  int subtotal = 268435456;
```

```
  int tax      = subtotal * 8 / 100;   // 8% sales tax
  int discount = subtotal / 3;         // "a third off"
  int total    = subtotal + tax - discount;

  std::printf("subtotal (cents)   = %d\n", subtotal);
  std::printf("tax  (8%%)          = %d   <- overflowed\n", tax);
  std::printf("discount (1/3)     = %d   <- truncated\n", discount);
  std::printf("total              = %d\n", total);
```

```
subtotal (cents)   = 268435456
tax  (8%)          = -21474836   <- overflowed
discount (1/3)     = 89478485   <- truncated
total              = 157482135

expected tax       = 21474836.48
expected discount  = 89478485.33
```

On an invoice of $2,684,354.56, `subtotal * 8` overflows and the tax comes out negative. The discount loses its fraction, and the total is meaningless. The same calculation in `inside`:

```
// Dollars in 1-cent steps, up to $10M. The scale is part of the type.
using money = inside<{{0, 10'000'000}, per<100>}, round_nearest>;

int main()
{
  money subtotal{2'684'354.56};

  // subtotal * 0.08 widens; nothing can overflow. Assigning back into
  // 'money' snaps to the nearest cent, because 'money' says round_nearest.
  auto  tax_exact = subtotal * 0.08_ins;
  money tax       = tax_exact;

  // A third of the subtotal is not representable in cents. It stays exact
  // until it is stored, and then rounds by the stated rule.
  auto  disc_exact = subtotal * just<1, 3>;
  money discount   = disc_exact;

  money total = subtotal + tax - discount;

  std::cout << "subtotal           = $" << subtotal << "\n";
  std::cout << "tax  (8%)          = $" << tax      << "   (exact, no overflow)\n";
  std::cout << "discount (1/3)     = $" << discount << "   (rounded to nearest cent)\n";
  std::cout << "total              = $" << total    << "\n";

  // The intermediate really is exact: a grid in thirds of a cent stores
  // the same value with nothing rounded away.
  using third_cent = inside<{{0, 10'000'000}, per<300>}, round_nearest>;
  third_cent discount_exact = disc_exact;

  std::cout << "\ndiscount on a 1/300 grid       = $" << discount_exact
            << "   (nothing was lost)\n";
```

```
subtotal           = $2684354.56
tax  (8%)          = $214748.36   (exact, no overflow)
discount (1/3)     = $894784.85   (rounded to nearest cent)
total              = $2004318.07

discount on a 1/300 grid       = $67108864/75   (nothing was lost)
```

- **The overflow is gone by construction.** `subtotal * 0.08_ins` widens to a grid that holds the product.
- **The tax is exact to the cent.** `0.08_ins` is an exact rational, not the nearest `double`. Storing into `money` rounds to nearest because the declaration says so.
- **The third stays exact until it is stored.** On a grid of thirds of a cent it is exactly 67108864/75 dollars. The rounding happened once, at storage, by a stated rule.
- **The friction is small and deliberate.** Literals need a grid (`0.08_ins`, `just<1, 3>`), and `money` needs a declaration. That declaration is where the range, scale and rounding rule get written down.

6.2 A sensor reading

Physical code adds units to the mix. Here a 12-bit ADC reading is converted to millivolts in `int`. It compiles cleanly at `-Wall -Wextra`:

```
int main()
{
  const int raw       = 2048;   // counts
  const int offset    = 12;     // counts
  const int gain      = 1241;   // counts per volt (data sheet: 1240.909)
  const int limit_mv  = 2000;   // over-voltage threshold, millivolts

  int mv = (raw - offset) * 1000 / gain;

  std::printf("reading            = %d mV   <- truncated\n", mv);
  std::printf("over-voltage?      = %s   <- counts compared with mV\n",
              raw > limit_mv ? "yes" : "no");

  std::printf("\nexpected           = 1641 mV (1640.73), no over-voltage\n");
```

```
reading            = 1640 mV   <- truncated
over-voltage?      = yes   <- counts compared with mV

expected           = 1641 mV (1640.73), no over-voltage
```

The gain was rounded to whole counts per volt, the division truncated, and the over-voltage check compares 2048 counts with 2000 millivolts. With quantities, counts get a base dimension of their own, as in `examples/adc_calibration.cpp`:

```
// ADC counts: a base dimension, its quantity and its unit.
inline constexpr base_dimension count_def{"count"};
inline constexpr quantity_spec  adc_counts_def{"adc_counts", count_def};
inline constexpr named_unit     cts_def{"cts", spec_c<adc_counts_def>};
inline constexpr auto           cts = unit_c<cts_def>;

using reading    = quantity<cts, inside<{0, 4095}>>;   // 12-bit ADC
using gain_t     = quantity<cts / si::volt, inside<{{1000, 1500}, per<1000>}>>;
using millivolts = quantity<si::milli(si::volt), inside<{-5000, 5000}, round_nearest>>;

template <class A, class B>
concept comparable = requires(A a, B b) { a < b; };

int main()
{
  const reading                          raw{2048};
  const quantity<cts, inside<{-64, 64}>> offset{12};
  const gain_t                           gain{rational{1240909, 1000}};
```

```
  const millivolts                        limit{2000};

  // The gain excludes 0, so the quotient is a plain, exact voltage.
  auto       volts = (raw - offset) / gain;
  millivolts mv    = value_cast<millivolts>(volts);   // one rounding

  std::cout << "reading            = " << mv << "\n";
  std::cout << "over-voltage?      = " << (mv > limit ? "yes" : "no") << "\n";

  // `raw > limit` does not compile: counts are not volts.
  static_assert(!comparable<reading, millivolts>);
```

```
reading            = 1641 mV
over-voltage?      = no
```

The gain keeps its data-sheet value exactly. Its range excludes 0, so `(raw - offset) / gain` is a plain, exact voltage, rounded once onto millivolts. Comparing a reading with a voltage is no longer a bug to find: it does not compile.

7 Correctness

7.1 Errors caught before the program runs

Grids are compile-time values and the arithmetic is `constexpr`, so whole categories of defect become build failures:

```
  // Range violations are knowable before the program exists.
  static_assert(conversion_overflows<pct>(150));
  static_assert(conversion_overflows<pct>(-1));
  static_assert(!conversion_overflows<pct>(50));

  // Saturating and wrapping casts are constant-evaluable.
  static_assert(clamp_cast<pct>(150) == 100);
  static_assert(wrap_cast<angle>(370) == 10);
  static_assert(wrap_cast<angle>(-10) == 350);

  // Arithmetic results are exact and checkable at compile time.
  constexpr pct a{60};
  constexpr pct b{55};
  static_assert(a + b == 115);   // widened past 100, and known to be so
```

Every line is a `static_assert`. An out-of-range constant is rejected outright:

```
$ g++ -std=c++26 -I ../../include x02_range_rejected.cpp
error: 'constexpr void beman::inside::policy<W,E>::report(beman::inside::errc) const'
       called in a constant expression
error: call to non-'constexpr' function 'void
    beman::inside::detail::raise(beman::inside::errc,
       const char*)'
```

The error path is deliberately not `constexpr`, so reaching it during constant evaluation is a compile error. Dimension and kind mismatches are caught the same way, as constraint failures with a stated reason (§5.7). The library's `tests/beman/inside/constexpr.test.cpp` relies on this: 159 `static_asserts` that run inside the compiler.

7.2 The error vocabulary

How a failure is reported depends on how you spell the call, and the shape matches the caller's role:

Shape	Spelling	When
Plain value	`a + b`	The type proves the operation cannot fail.
`expected<T,errc>`	`a / b`, `try_make`, `to<T>()`, `math::tan`	It can fail at runtime. A math argument outside the domain for the whole input grid is a compile error instead.
Error code	`x.policy(ec) = v`	Hot paths; the first error sticks.
Throwing	default (`checked`)	Programming errors not handled locally.
Callback	`on_clamp`, `on_wrap`	The event is expected and carries data.

Quantities keep these shapes: dividing by a quantity whose grid holds 0 gives an `expected`, as it does for an `inside`. The codes form a small closed set (`detail/debug.hpp:30`). A value that does not fit is always `overflow`, whichever path found it:

```
enum class errc { domain_error, division_by_zero, overflow,
                  rounding_error, not_finite, invalid_format };
```

There is no `<system_error>` dependency. `set_error_handler` replaces the failure path where exceptions are unavailable, which lets the library run freestanding. Callbacks matter because some "errors" are the domain's semantics. Seconds overflowing 59 is a carry:

```
void add_seconds(auto n)
{
  seconds.on_wrap([&](auto, auto carry) { add_minutes(carry); }) += n;
}
```

That is the whole carry logic of `examples/clock.cpp`.

7.3 Exactness

The notch is an exact rational, so cents and thirds have no approximation. Here is the money example from §3:

```
  // 0..$1,000,000 in 1-cent steps. 0.01 is not exact in binary, so the
  // notch is given as an exact rational -- not as 0.01.
  using money = inside<{{0, 1'000'000}, per<100>}, round_nearest>;
  static_assert(sizeof(money) == 4);   // still four bytes

  money total{0};
  for (int i = 0; i < 10000; ++i)
    total += money{0.01};

  std::cout << "10000 x $0.01      = $" << total << "\n";
  std::cout << "exactly $100.00    ? "
            << (total == money{100.00} ? "true" : "false") << "\n";

  // Read the value back out exactly, as an integer pair.
  std::cout << "as a fraction      = " << total.numerator()
            << "/" << total.denominator() << "\n";
```

```
10000 x $0.01      = $100.00
exactly $100.00    ? true
as a fraction      = 100/1
```

The result is exactly one hundred dollars, in four bytes. The same computation drifted by 1.4×10^{-11} in `double`. Exactness needs no arbitrary-precision arithmetic, because the grid is known at compile time and the value is an integer count of notches. Nothing is stored as floating point. A `double` enters through one exact rounding onto the grid and leaves through one correctly rounded conversion.

7.4 Reproducibility

For integer and rational storage, `+ - * /` reproduce unconditionally, because no floating point is involved. Conversion from `double` decodes the IEEE-754 bits and needs no FPU.

`beman::inside::math` returns the *correctly rounded* point of its output grid, under the output's rounding mode. That result depends only on the input and the grid, so it is the same at compile time, at runtime, and with or without an FPU. Each result is computed with a proved error bound and escalated (Ziv's strategy) until the bound puts it in one slot. Most calls are answered by cheap tiers: compile-time tables for small grids, and the library's own `double` and double-`double` kernels with proved bounds. An integer path answers the rest, and it is the only path at compile time and without an FPU. C++26 makes most of `<cmath>` `constexpr` but leaves its accuracy unspecified, so a `std::sin` folded by the compiler need not match the one computed at runtime. Against a long-double reference, every function's maximum error is 0.5000 notches (`docs/accuracy.md`). `-ffast-math` builds are refused with an `#error`. Correct rounding costs time, which §8.5 measures.

8 Performance

The cost of `inside` is not a number. It is **a range with a negative end**:

Regime	When	Measured
Negative cost	A chain of operations whose intermediate ranges the grid arithmetic discharges; comparisons that run on raw storage.	21% faster than native checked (§8.3); scalar comparison at 130% of native
Zero cost	Integer operands, where all the range reasoning happens in the type system.	Byte-identical disassembly; a unit conversion is one `mov` (§8.1); construct, add, `std::accumulate`, sort, transform at 98–100%
Additional cost	Continuous (rational) grids, element-wise checking, transcendentals correctly rounded at full `double` resolution.	rational add about 5× `double`; checked loop at 20%; fine math 4–9× `std`

How the code is written decides the regime more than the library does. Code that stays in `inside` lets the grids discharge obligations. Code that round-trips through raw `int` at every step pays a check at each boundary. Any summary of the form "`inside` costs N%" is wrong in at least one direction.

8.1 The fast path is byte-identical to native

```
using val = inside<{0, 1000}>;

// The inside version.
[[gnu::noinline]] auto inside_add(val a, val b)
{
  return a + b;   // inside<{0, 2000}>: the sum widens, nothing to check
```

```
}

// The native version, for comparison.
[[gnu::noinline]] int native_add(int a, int b)
{
  return a + b;
}

// A unit conversion rescales the grid and keeps the raw.
using km = units::quantity<units::si::kilo(units::si::metre), inside<{{0, 100}, per<1000>}>>;

[[gnu::noinline]] auto to_mm(km d)
{
  return d.in(units::si::milli(units::si::metre));
}
```

```
$ g++ -std=c++26 -O2 -I ../../include -c p01_codegen.cpp && objdump -d p01.o

0000000000000000 <inside_add(inside<...>, inside<...>)>:
   0:   endbr64
   4:   lea    (%rdi,%rsi,1),%eax
   7:   ret

0000000000000010 <native_add(int, int)>:
  10:   endbr64
  14:   lea    (%rdi,%rsi,1),%eax
  17:   ret

0000000000000020 <to_mm(quantity<...>)>:
  20:   endbr64
  24:   mov    %edi,%eax
  26:   ret
```

`inside_add` and `native_add` are byte-identical under the default `checked` policy: the sum widens to $[0, 2000]$, so there is nothing to check. The km-to-mm conversion is a single `mov`: it rescales the grid and keeps the raw. An in-place `+=` can leave its range, so a `checked` element is checked on every pass and the loop stays scalar, as in the accumulate table below. With `unsafe` it vectorises, on 16-bit lanes (its storage for $[0, 1000]$) where native uses 32-bit lanes. `tests/beman/inside/check_codegen.sh` runs in CI. It disassembles named kernels and fails on any `call` other than the cold error report, and on a unit conversion that does arithmetic. Whether the loop vectorises is reported, not gated.

8.2 The like-for-like comparison

The fastest row in every table is plain, unchecked native code. It is fastest because it does not do the job: it is the code of §2. The honest baselines are native code that enforces the same range, written as a competent programmer would write it (`tests/beman/inside/bench.test.cpp:62`):

- `native clamped`: `std::clamp(v, 0, 200)`;
- `native checked`: a hand-rolled bounded struct that throws on violation.

`p02_fair_bench.cpp` measures all five contenders on the same data: a summed pass over 4096 values, repeated 20,000 times, with no throws.

Implementation	ns/op	vs. unchecked	enforces?	vs. safe native
`native unchecked`	0.439	100.0%	no	—
`native clamped`	0.459	95.6%	yes	100.0%
`native checked`	0.518	84.6%	yes	88.5%
`inside<checked>`	0.522	84.1%	yes	**87.9%**
`inside<unsafe>`	0.439	100.0%	no	—

Mean of three runs pinned to one core; GCC 15.2.0, `-std=c++26 -O2`, x86-64.

The guarantee itself costs 5–15% in plain C++. `inside<checked>` runs at 99.4% of the hand-rolled checked struct, so the library adds under 1% on top. `inside<unsafe>` matches unchecked native. This is a favourable pattern: a tight, predictable loop. It shows that the gap is small *when you compare against code doing the same work*, not that `inside` is free everywhere.

8.3 Where are the checks?

That benchmark gives `inside` nothing to prove. Every iteration converts a raw `int` in and narrows the result back out, and that is the one shape where a check cannot be avoided. Disassembling each step shows where checks appear:

Operation	Emitted	Check?
`bchecked(int v)`	`cmp $0xc8,%edi; ja ...`	yes
`auto c = a + b` (kept wide)	`movzbl; movzbl; lea; ret`	**none**
`bchecked c = a + b`	`add; cmp $0xc8,%rax; jg ...`	yes
`checked_u8 c = a + b` (native)	`add; cmp $0xc8,%eax; jg ...`	yes
`bchecked b = a` (same grid)	`mov; ret`	**none**
`wide w = a` (widening)	`movzbl; ret`	**none**

Checks appear in only two places: entering from a raw integer, and narrowing into a smaller grid. The narrowing check is the same instructions as the native struct's. A native checked type must re-validate after *every* step of a chain. `inside` lets intermediates widen and checks only the final store. `p03_check_elision.cpp` sums four values in $[0, 50]$ into $[0, 200]$:

Implementation	ns/op	vs. unchecked	runtime checks
`native unchecked`	0.874	100.0%	0
`native checked`	1.067	81.9%	3
`inside<checked>`	0.881	**99.2%**	**0**

`inside<checked>` is **21% faster than the native checked type** and within noise of no checking at all. The sum provably fits $[0, 200]$, so the function is the same two `add`s and an `lea` as the unchecked version. The native one has three `cmp`/`jg` pairs and spills. If the result type is narrowed to $[0, 199]$, the store check (`cmp $0xc7,%al; ja`) reappears at once. Hand-written checks cannot be removed, because nothing in `int` records what has already been established.

8.4 The generated tables

These come from `docs/performance.md` (GCC 15, C++26, `-O3`, one run pinned to one core). *rel.* is throughput relative to the first row, so smaller means slower. Frequency scaling was active, so read the ratios and instruction counts, not the nanoseconds. Sub-nanosecond rows move by 15–20% from run to run.

construct (u8 [0,200])

rel.	ns/op	ins/op	enforces?	implementation
100.0%	0.29	7.00	no	`native uint8`
62.8%	0.47	13.00	yes	`native clamped`
90.1%	0.33	9.00	yes	`native checked (exceptions)`
99.9%	0.29	7.00	no	`inside<unsafe>`

add (u8 [0,200])

rel.	ns/op	ins/op	enforces?	implementation
100.0%	0.41	11.00	no	`native uint8`
49.7%	0.82	15.00	yes	`native clamped`
79.8%	0.51	17.00	yes	`native checked (exceptions)`
99.7%	0.41	10.00	no	`inside<unsafe>`

accumulate 1000 (u32 [0,200000], per element)

rel.	ns/op	ins/op	implementation
100.0%	0.05	1.03	`native loop`
100.1%	0.05	1.03	`inside<unsafe> loop`
20.1%	0.26	9.01	`inside<checked> loop`
47.4%	0.11	2.47	`beman::inside::sum<checked> (bulk check)`
100.0%	0.05	1.03	`std::accumulate native`
100.0%	0.05	1.03	`std::accumulate inside`

A checked element-wise loop checks every element and so cannot vectorise. `beman::inside::sum` checks the total instead and is 2.4× faster. Without the check, both loops vectorise like native code.

comparison ==

rel.	ns/op	ins/op	implementation
100.0%	0.53	12.00	`native int16`
101.3%	0.52	12.00	`inside same grid`
130.0%	0.41	9.00	`inside == int scalar (value raw)`
129.6%	0.41	9.00	`inside Q8.8 == int scalar (index raw)`

The grid lets a comparison against a scalar run on raw storage, which takes fewer instructions than native.

rational add (continuous grid)

rel.	ns/op	ins/op	implementation
100.0%	4.97	97.20	`native fraction`
863.6%	0.58	12.00	`std double (reference)`
187.1%	2.66	62.27	`inside (continuous)`

Overflow-checked rational addition beats a hand-rolled reduced `int64` fraction, and costs about 5× a `double`. That is the price of exactness. Only continuous grids pay it: any notched grid, thirds included, stores an integer index. Sort and `std::transform` run at 98.7% and 99.0% of native.

Increment (`u32 [0,200000]`) is the outlier, at 12.0%, and the library's arithmetic is not the cause. As plain functions, the two increments compile identically (`lea; xor; cmp; cmovae`). In the benchmark, GCC gives the native counter a predicted branch and the `inside` one a conditional move on a loop-carried dependency through memory.

8.5 What correct rounding costs

The cost of `beman::inside::math` follows the output grid. The report measures it onto the grid deduced from a 2^{-14} input:

Function (deduced grid)	`std` ns	`inside` ns	rel.
`sin`	3.72	4.67	79.8%
`hypot`	2.97	3.14	94.5%
`atan2`	7.70	7.56	101.7%
`log`	2.61	5.37	48.6%
`sqrt`	1.23	2.20	55.6%
`exp`	2.87	5.37	53.5%

This flatters the library. `std` computes 53 bits, while the deduced grids need 15 bits for `sin`, 12 for `log`, 17 for `sqrt` and 26 for `exp`. `p04_math_cost.cpp` also measures output grids as fine as a `double`:

Function	`std` ns	deduced grid ns	`double`-fine ns	fine vs. `std`
`sin` (2^{-52})	3.40	4.93	14.5	4.3×
`exp` (2^{-40} on $[0, 4096]$)	2.67	5.61	12.4	4.7×
`log` (2^{-48})	2.32	5.57	17.4	7.5×
`sqrt` (2^{-51})	1.23	2.38	10.5	8.6×

Mean of three runs pinned to one core; GCC 15.2.0, `-std=c++26 -O3 -mfma`, x86-64.

At full `double` resolution, correct rounding costs **4 to 9**× `std`. In return, the results are correct to the last bit and identical on every platform, compiler and optimisation level, with or without an FPU. Ask for the output grid you need, not the finest one available.

The imported tables are a single run on one machine. §8.2, §8.3 and §8.5 are means of three runs. All of them establish orders of magnitude, not precise ratios. To decide on the margin, rerun `perf_report` and the `p0*` programs on your own hardware.

9 When not to use this

- **It is not yet ready for production.** Version 0.3 is at the Beman "under development" maturity level, and the API may change.
- **It requires C++23**: GCC 14+, Clang 19+ (Clang 18 with libc++), or MSVC 19.51+. Grids past 64 bits need C++26 static reflection, which today means GCC 16 with `-freflection`.
- **Compile times increase.** Arithmetic on grids is compile-time work, and it happens in your build. Units add about 7 ms per expression on top (`docs/units.md`).
- **The units layer is narrower than mp-units.** There are no `double` representations, no vector or tensor quantities and no logarithmic units, and the kind rules are a simplified version of mp-units'.
- **Error messages are template errors.** On the guided path a hand-written message names the fix (§5). Off it, you get the usual template output.
- **Widening surprises `auto` users.** `a + b` does not have the type of `a`, so `auto` variables in a chain get progressively wider types. That is correct, but most C++ programmers do not expect it.
- **Correct rounding is not free.** At `double` resolution it costs 4–9× `<cmath>`. Code that needs `double` throughput and tolerates `double` error should call `<cmath>`.

- **Not everything needs it.** A loop counter is not where numeric bugs come from. The machinery pays off where a wrong answer is expensive: money, physical units, fixed-point signal processing, registers, control loops, and anything that crosses a system boundary.

10 Conclusion

The defects of §2 are not exotic. They are the arithmetic of the type C++ programmers reach for by default, and carelessness does not cause them. The midpoint bug lived nine years in the JDK, and Ariane 5 was destroyed by code that had been correct on the previous rocket. These defects persist because the information needed to catch them is absent from the program:

- `int` does not know its range, so it cannot check it.
- It does not know what it counts, so it cannot object to a mismatch.
- It has no rounding policy, so it truncates.
- It has no representation for failure, so division by zero kills the process.
- Because overflow is undefined, the compiler may assume it never happens, and a program that should print `3` becomes a `main` with no instructions.

`inside` shows that a range, a step size, a unit and an edge policy written into the type are enough to fix these. Overflow becomes unreachable, division by zero becomes a value, rounding becomes a decision, and a unit mismatch becomes a compile error. On the fast paths, it emits the same instructions as the `int` it replaces.

The unit layer stands on mp-units' design, and the number beneath it is what makes its conversions exact. For any number in your program, you already know its range, its units and what should happen at its limits. Writing that down costs a line. Not writing it down costs whatever the bug costs.

A Building the examples

`docs/int-considered-harmful/README.md` lists how to build every program. The `h*` and `d*` programs need only a compiler. The others need `-I ../../include`, and `b11` needs `g++-16 -std=c++26 -freflection`. The two `x*` files are meant to fail to compile. The library itself builds with CMake 3.30+:

```
cmake --preset gcc-release && cmake --build --preset gcc-release
ctest --preset gcc-release
ctest --preset gcc-release -L example    # run the 38 example programs
```

B Program index

File	Demonstrates
`h01_overflow_ub.cpp`	Overflow check folded away, even at `-O0`
`h02_overflow_loop.cpp`	`main` compiled to no instructions
`h03_intmin.cpp`	`-INT_MIN` and `abs(INT_MIN)` negative
`h04_divzero.cpp`	`SIGFPE` on `x/0` and `INT_MIN/-1`
`h05_signed_unsigned.cpp`	Loop over an empty container
`h06_promotion.cpp`	Promotion, `char` signedness, `int` width
`h07_truncdiv.cpp`	Truncation toward zero, negative remainder
`h08_shift.cpp`	Defined shifts that mislead; a count past the width
`h09_units.cpp`	Cents added to dollars
`h10_binsearch.cpp`	Midpoint outside the range
`h11_narrowing.cpp`	Silent narrowing (Ariane 5)
`d01_decimal.cpp`	`0.1 + 0.2 != 0.3`
`d02_money_drift.cpp`	Accumulated drift on a running total
`d03_assoc.cpp`	Addition is not associative
`d04_precision.cpp`	2^{53}; catastrophic cancellation
`d05_nan.cpp`	`NaN` breaks equality and ordering
`b00_declarations.cpp`	Quantities: %, EUR/ct, ° ↔ rad, SPS
`b01_no_overflow.cpp`	Widening arithmetic; the midpoint, fixed
`b02_divzero.cpp`	Division as `expected`; exact by default
`b03_policies.cpp`	`checked` / `clamp` / `wrap` / `try_make`
`b04_money.cpp`	Exact cents in four bytes
`b05_units.cpp`	Scale in the type
`b06_compile_time.cpp`	Range errors as build failures
`b07_rounding.cpp`	Every rounding mode honoured
`b08_index.cpp`	Bounded indices
`b10_negation.cpp`	Negation reflects the grid
`b11_no_ceiling.cpp`	Grids past 64 bits (C++26)
`b12_impulse.cpp`	lbf·s + N·s converts; impulse + length rejected
`b13_quantities.cpp`	Conversions, points, kinds, ratios
`m_before.cpp` / `m_after.cpp`	The worked migration: an invoice
`m2_before.cpp` / `m2_after.cpp`	The worked migration: an ADC reading
`x01_bare_int_rejected.cpp`	Bare literal rejected (expected failure)
`x02_range_rejected.cpp`	Out-of-range constant rejected (expected failure)
`p01_codegen.cpp`	Fast path and unit conversion vs. native
`p02_fair_bench.cpp`	Like-for-like benchmark vs. checked native
`p03_check_elision.cpp`	Chain of ops: inside elides checks native cannot
`p04_math_cost.cpp`	What correct rounding costs, coarse and fine

C References

1. E. W. Dijkstra, "Go To Statement Considered Harmful," *Communications of the ACM*, 11(3):147–148, March 1968.

2. J. Bentley, *Programming Pearls*, Addison-Wesley, 1986 — the binary search midpoint.

3. J. Bloch, "Extra, Extra — Read All About It: Nearly All Binary Searches and Mergesorts are Broken," Google Research Blog, June 2006.

4. J. L. Lions et al., *ARIANE 5 Flight 501 Failure: Report by the Inquiry Board*, ESA/CNES, July 1996.

5. NASA, *Mars Climate Orbiter Mishap Investigation Board Phase I Report*, November 1999.

6. D. Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic," *ACM Computing Surveys*, 23(1):5–48, 1991.
7. ISO/IEC 14882, *Programming languages — C++* (C++26): [expr.pre] (undefined behaviour on overflow), [expr.shift], [expr.mul], [conv.integral].
8. M. Pusz, *mp-units*, https://github.com/mpusz/mp-units.
9. M. Pusz et al., *Quantities and units library*, WG21 P3045, 2024–2026.
10. M. C. Schabel and S. Watanabe, *Boost.Units*, https://www.boost.org/libs/units.
11. A. Ziv, "Fast evaluation of elementary mathematical functions with correctly rounded last bit," *ACM Transactions on Mathematical Software*, 17(3):410–423, 1991.
12. The `inside` repository, https://github.com/PeterNeiss/inside: `README.md`, `docs/tutorial.md`, `docs/policies.md`, `docs/arithmetic.md`, `docs/storage.md`, `docs/units.md`, `docs/math.md`, `docs/determinism.md`, `docs/accuracy.md`, `docs/performance.md`.